

1. Sistemas Operativos - Generalidades

1.a) ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

El encargado de administrar el tiempo de CPU es el scheduler del sistema operativo. Su función principal es decidir qué tarea debe ejecutarse en cada instante. En FreeRTOS, el scheduler selecciona la tarea de mayor prioridad que se encuentre en estado Ready.

1.b) Explique a qué se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?

El contexto de ejecución es toda la información que permite pausar una tarea y luego retomarla exactamente desde el punto en el que fue interrumpida. Cuando una tarea sale del estado Running, FreeRTOS guarda el estado necesario para que pueda continuar más adelante como si nunca hubiese sido interrumpida. Entre las variables intervinientes se encuentran los registros del procesador, el program counter, el stack pointer, la pila propia de la tarea, su prioridad, su estado y la estructura de control de la tarea.

1.c) ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Un sistema operativo de tiempo real es aquel que permite organizar el software para responder a eventos dentro de tiempos definidos. No alcanza con que el resultado sea correcto: también importa cuándo se obtiene. Se usa un RTOS porque permite estructurar el firmware en tareas concurrentes, asignar prioridades, bloquear tareas que no tienen trabajo pendiente, atender eventos mediante mecanismos de sincronización y lograr una temporización más ordenada que en un programa bare-metal resuelto con un único while(1), especialmente cuando existen periféricos, comunicación, HMI y control periódico.

2. Kernel de FreeRTOS

2.a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

En FreeRTOS una tarea es una función en C que representa un hilo de ejecución independiente. Normalmente posee un punto de entrada, se implementa con un bucle infinito y no debe retornar. Cada instancia creada de una tarea tiene su propia pila, su propio contexto y puede recibir un parámetro void * al momento de ser creada.

```
#include "FreeRTOS.h"
#include "task.h"

static void vBlinkTask(void *pvParameters)
{
    for(;;)
    {
        /* Código de la tarea: alternar un GPIO, leer un sensor, etc. */
        toggle_led();
        vTaskDelay(pdMS_TO_TICKS(500));      /* La tarea queda bloqueada 500 ms */
    }

    vTaskDelete(NULL);      /* No debería alcanzarse */
}

int main(void)
{
    xTaskCreate(vBlinkTask,
                "Blink",
                1024,
```

```

        NULL,
        1,
        NULL) ;

vTaskStartScheduler() ;

}

```

2.b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Los estados principales son Running, Ready, Blocked y Suspended. Running es la tarea que está usando la CPU. Ready es una tarea lista para ejecutar, pero que no fue elegida porque hay otra de igual o mayor prioridad. Blocked es una tarea esperando un evento temporal o de sincronización. Suspended es una tarea excluida del scheduler hasta ser reanudada explícitamente. Las transiciones se producen por acción del scheduler, por la interrupción de tick, por funciones de bloqueo como vTaskDelay(), vTaskDelayUntil(), xQueueReceive(), xQueueSend(), xSemaphoreTake(), por eventos que desbloquean como xQueueSend(), xQueueReceive(), xSemaphoreGive(), xSemaphoreGiveFromISR(), y por suspensión o reanudación mediante vTaskSuspend(), vTaskResume() y vTaskResumeFromISR().

2.c) ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿Y a qué nos referimos con el concepto de “tareas manejadas por eventos”?

Una tarea de procesamiento continuo es aquella que siempre está disponible para ejecutarse porque no se bloquea esperando eventos. Si tiene una prioridad alta, puede impedir que tareas de menor prioridad se ejecuten. En cambio, una tarea manejada por eventos solo trabaja cuando ocurre el evento que la despierta: recepción de datos en una cola, llegada de un semáforo, vencimiento de un delay periódico, interrupción diferida, etc. Este segundo enfoque es el más adecuado en un RTOS porque evita desperdiciar CPU.

2.d) ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Al entrar al estado bloqueado, una tarea puede esperar eventos temporales o eventos de sincronización. Los eventos temporales son esperas por tiempo relativo o absoluto, por ejemplo vTaskDelay(10 ms) o vTaskDelayUntil() para lograr periodicidad estable. Los eventos de sincronización provienen de otras tareas o interrupciones, por ejemplo datos disponibles en una cola, semáforos, mutex o notificaciones.

2.e) ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

La Idle Task es creada automáticamente por FreeRTOS al iniciar el scheduler. Su función es asegurar que siempre exista al menos una tarea disponible para ejecutar y realizar tareas internas, como liberar memoria de tareas eliminadas. El procesador no queda “sin ejecutar ninguna tarea”: si no hay tareas de usuario listas, ejecuta la Idle Task. En aplicaciones de bajo consumo, el hook de idle puede usarse para dormir el microcontrolador.

3. Almacenamiento de datos en colas

3) Desarrolle cómo es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Una cola de FreeRTOS es un objeto del kernel capaz de almacenar una cantidad finita de elementos de tamaño fijo. Al crearla con xQueueCreate(uxQueueLength, uxItemSize) se define cuántos elementos puede contener y cuántos bytes ocupa cada elemento. El almacenamiento normal es FIFO: los datos se escriben al final de la cola con xQueueSendToBack() y se leen desde el frente con xQueueReceive(). También existe la posibilidad de insertar al frente con xQueueSendToFront(). FreeRTOS copia byte a byte el dato enviado hacia el área interna de almacenamiento de la cola. Por eso, si una tarea modifica su variable local después de enviarla, el valor que está

dentro de la cola no cambia: la cola guardó una copia. Al recibir, FreeRTOS copia el elemento desde la cola hacia el buffer indicado por la tarea receptora y, en `xQueueReceive()`, lo elimina de la cola.

4. Múltiples tareas escritoras y una sola tarea lectora

4) Explique utilizando un ejemplo con código por qué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

Es común que una cola tenga múltiples escritoras y una sola lectora porque muchas partes del sistema pueden producir eventos, comandos o mediciones, pero conviene centralizar el procesamiento en una tarea consumidora. Por ejemplo, una tarea UART, una tarea HMI y una tarea de seguridad pueden escribir comandos en una misma cola `ControlQueue`, mientras que la tarea de control del motor es la única que los interpreta.

El bloqueo por lectura ocurre cuando una tarea intenta recibir desde una cola vacía y especifica un tiempo de espera distinto de cero. En ese caso la tarea pasa a `Blocked` hasta que otra tarea o una ISR escriba un dato, o hasta que venza el `timeout`. El bloqueo por escritura ocurre cuando una tarea intenta escribir en una cola llena y especifica un tiempo de espera; la tarea queda bloqueada hasta que se libere espacio o expire el tiempo.

```
typedef enum { CMD_SET_ANGLE, CMD_STOP, CMD_SET_PROFILE } CommandId_t;

typedef struct
{
    CommandId_t id;
    int32_t value;
} Command_t;

QueueHandle_t xControlQueue;

static void vUartTask(void *pvParameters)
{
    Command_t cmd;
    for(;;)
    {
        cmd = uart_read_command();
        /* Escritora 1: si la cola está llena, espera hasta 20 ms. */
        xQueueSendToBack(xControlQueue, &cmd, pdMS_TO_TICKS(20));
    }
}

static void vHmiTask(void *pvParameters)
{
    Command_t cmd;
    for(;;)
    {
        cmd = hmi_read_command();
        /* Escritora 2: también produce comandos para la misma cola. */
        xQueueSendToBack(xControlQueue, &cmd, pdMS_TO_TICKS(20));
    }
}

static void vControlTask(void *pvParameters)
{
    Command_t rx;
    for(;;)
    {
        /* Lectora única: se bloquea hasta recibir un comando. */
        if(xQueueReceive(xControlQueue, &rx, portMAX_DELAY) == pdPASS)
```

```

        {
            process_motor_command(rx);
        }
    }
}

void app_main(void)
{
    xControlQueue = xQueueCreate(8, sizeof(Command_t));
    xTaskCreate(vUartTask, "UART", 2048, NULL, 2, NULL);
    xTaskCreate(vHmiTask, "HMI", 2048, NULL, 1, NULL);
    xTaskCreate(vControlTask, "CONTROL", 2048, NULL, 3, NULL);
}

```

En este ejemplo la cola desacopla productores y consumidor. Las tareas escritoras no necesitan saber cuándo se ejecutará el control; solo depositan mensajes. La tarea lectora puede tener mayor prioridad para reaccionar rápido cuando aparece un comando.

5. Envío de datos compuestos y datos grandes

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?

Para enviar datos compuestos se define una estructura y se crea la cola con `sizeof(estructura)`. Esto no entra en conflicto con la definición del recurso, porque la cola no interpreta el contenido del dato: solo sabe cuántos bytes debe copiar por elemento. Mientras todos los envíos a esa cola usen el mismo tipo y tamaño de dato, el comportamiento es consistente.

```

typedef struct
{
    uint8_t source;
    uint8_t command;
    int32_t dataValue;
} Data_t;

QueueHandle_t xDataQueue;

void app_main(void)
{
    xDataQueue = xQueueCreate(10, sizeof(Data_t));
}

void sender(void)
{
    Data_t tx = { .source = 1, .command = 3, .dataValue = 1200 };
    xQueueSendToBack(xDataQueue, &tx, 0); /* Se copian sizeof(Data_t) bytes */
}

```

Si los datos fueran grandes, copiar toda la estructura en la cola consumiría más RAM y más tiempo de CPU. En ese caso conviene enviar por la cola un puntero a un bloque de memoria o a un buffer estático, y no el bloque completo. La cola copia solo el puntero, que suele ocupar 4 bytes en un microcontrolador de 32 bits.

```

typedef struct
{
    uint8_t frame[256];
    size_t length;
} BigFrame_t;

```

```

QueueHandle_t xFramePtrQueue;

void app_main(void)
{
    /* La cola almacena punteros a BigFrame_t, no estructuras completas. */
    xFramePtrQueue = xQueueCreate(4, sizeof(BigFrame_t *));
}

void sender_big_frame(void)
{
    static BigFrame_t frameBuffer;
    BigFrame_t *pFrame = &frameBuffer;

    fill_frame(pFrame);
    xQueueSendToBack(xFramePtrQueue, &pFrame, 0);
}

void receiver_big_frame(void)
{
    BigFrame_t *pRx;

    if(xQueueReceive(xFramePtrQueue, &pRx, portMAX_DELAY) == pdPASS)
    {
        process_frame(pRx);
    }
}

```

La precaución importante es la vida útil del dato apuntado. No se debe enviar un puntero a una variable local de pila que dejará de existir al salir de la función. Se debe usar memoria estática, memoria dinámica correctamente gestionada, un pool de buffers o un mecanismo equivalente.

6. Funcionamiento del esquema de la Fig. 1

6) Explicar el funcionamiento del esquema de la Fig. 1.



Fig. 1. Recibir datos de múltiples fuentes

La Fig. 1 muestra un caso típico de una cola utilizada para recibir datos desde múltiples fuentes. Hay varias tareas productoras, por ejemplo CAN bus Task, Another Task y HMI Task. Todas generan mensajes con distinto significado y los envían hacia una misma cola. Del otro lado hay una tarea Controller que actúa como consumidora única.

Para que la tarea receptora pueda distinguir el origen o el significado de cada dato, no alcanza con enviar solo un número. Por eso se usa una estructura con campos como eDataID y lDataValue. El identificador eDataID indica qué representa el dato, por ejemplo velocidad del motor o setpoint de velocidad, y lDataValue contiene el valor asociado.

La tarea Controller recibe cada estructura, analiza el identificador y actualiza la variable correspondiente. Este patrón evita tener varias colas distintas cuando todos los datos deben terminar en el mismo módulo de control. Además, mantiene desacopladas a las tareas productoras respecto de la lógica interna del controlador.

7. Semáforos Mutex

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

Un mutex es un semáforo binario usado para exclusión mutua. Funciona como un token asociado a un recurso compartido: una tarea debe tomar el mutex antes de usar el recurso y devolverlo al finalizar. A diferencia de un semáforo usado para sincronización, un mutex tomado para proteger un recurso siempre debe ser liberado.

El siguiente ejemplo protege un LCD conectado por I2C. Dos tareas quieren escribir mensajes, pero solo una puede acceder al LCD a la vez. Sin mutex podrían mezclarse caracteres de ambas tareas y corromper la salida.

```
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

static SemaphoreHandle_t xLcdMutex;

static void lcd_print_safe(const char *msg)
{
    if(xSemaphoreTake(xLcdMutex, portMAX_DELAY) == pdTRUE)
    {
        lcd_clear();
        lcd_write_string(msg);
        xSemaphoreGive(xLcdMutex);    /* El mutex siempre se devuelve */
    }
}

static void vStatusTask(void *pvParameters)
{
    for(;;)
    {
        lcd_print_safe("Estado: OK");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

static void vAlarmTask(void *pvParameters)
{
    for(;;)
    {
        if(alarm_condition())
        {
            lcd_print_safe("ALARMA MOTOR");
        }
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

void app_main(void)
{
    lcd_init();
    xLcdMutex = xSemaphoreCreateMutex();

    if(xLcdMutex != NULL)
    {
```

```

        xTaskCreate(vStatusTask, "Status", 2048, NULL, 1, NULL);
        xTaskCreate(vAlarmTask, "Alarm", 2048, NULL, 2, NULL);
    }
}

```

La tarea que toma el mutex tiene acceso exclusivo al LCD. Si otra tarea intenta tomarlo mientras está ocupado, queda bloqueada hasta que el mutex sea devuelto. Así se evita que dos tareas accedan simultáneamente al mismo periférico.

8. Inversión de prioridad

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad” como se ve en la Fig. 2. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Respuesta: La inversión de prioridad ocurre cuando una tarea de alta prioridad queda esperando un recurso retenido por una tarea de baja prioridad. El problema se agrava si una tarea de prioridad media, que no necesita el recurso, se ejecuta e impide que la tarea de baja prioridad avance y libere el recurso. En consecuencia, la tarea de alta prioridad termina indirectamente esperando a una tarea de prioridad media.

En el ejemplo se usa un mutex para un recurso del microcontrolador, el bus I2C/LCD, y una cola para enviar comandos de impresión. Hay tres tareas para que el fenómeno sea evidente: baja prioridad toma el mutex y tarda en liberar; alta prioridad recibe comandos por cola y necesita el mismo mutex; media prioridad hace trabajo de CPU y retrasa a la baja.

```

typedef struct
{
    char text[32];
} LcdCmd_t;

static QueueHandle_t xLcdQueue;
static SemaphoreHandle_t xI2CMutex;

static void vLowPriorityTask(void *pvParameters)
{
    for(;;)
    {
        xSemaphoreTake(xI2CMutex, portMAX_DELAY);
        /* Simula una operación larga sobre el periférico I2C. */
        lcd_write_string("LP usando LCD");
        busy_wait_ms(300); /* Mala práctica: mantiene el mutex demasiado */
        xSemaphoreGive(xI2CMutex);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

static void vHighPriorityTask(void *pvParameters)
{
    LcdCmd_t cmd;

    for(;;)
    {
        /* La cola despierta a la tarea HP cuando llega un comando crítico. */
        if(xQueueReceive(xLcdQueue, &cmd, portMAX_DELAY) == pdPASS)
        {
            /* Si LP tiene el mutex, HP queda bloqueada aunque sea más prioritaria. */
            xSemaphoreTake(xI2CMutex, portMAX_DELAY);
            lcd_write_string(cmd.text);
        }
    }
}

```

```

        xSemaphoreGive(xI2CMutex);
    }
}

static void vMediumPriorityTask(void *pvParameters)
{
    for(;;)
    {
        /* Trabajo que no usa el LCD, pero puede adelantar a LP. */
        do_cpu_work();
        taskYIELD();
    }
}

static void vCommandProducerTask(void *pvParameters)
{
    LcdCmd_t cmd = { "HP necesita LCD" };

    for(;;)
    {
        vTaskDelay(pdMS_TO_TICKS(100));
        xQueueSendToBack(xLcdQueue, &cmd, 0);
    }
}

void app_main(void)
{
    xLcdQueue = xQueueCreate(4, sizeof(LcdCmd_t));
    xI2CMutex = xSemaphoreCreateMutex();

    xTaskCreate(vLowPriorityTask,    "LP",    2048, NULL, 1, NULL);
    xTaskCreate(vMediumPriorityTask, "MP",    2048, NULL, 2, NULL);
    xTaskCreate(vHighPriorityTask,   "HP",    2048, NULL, 3, NULL);
    xTaskCreate(vCommandProducerTask, "PROD", 2048, NULL, 2, NULL);
}

```

Secuencia: 1) LP toma el mutex del I2C/LCD. 2) Llega un comando a la cola y HP se desbloquea. 3) HP intenta tomar el mutex, pero no puede porque lo tiene LP, entonces queda bloqueada. 4) MP, al tener más prioridad que LP, se ejecuta y retrasa a LP. 5) HP no puede continuar hasta que LP logre ejecutar y devolver el mutex. Esa espera indirecta es la inversión de prioridad.

9. Herencia de prioridad

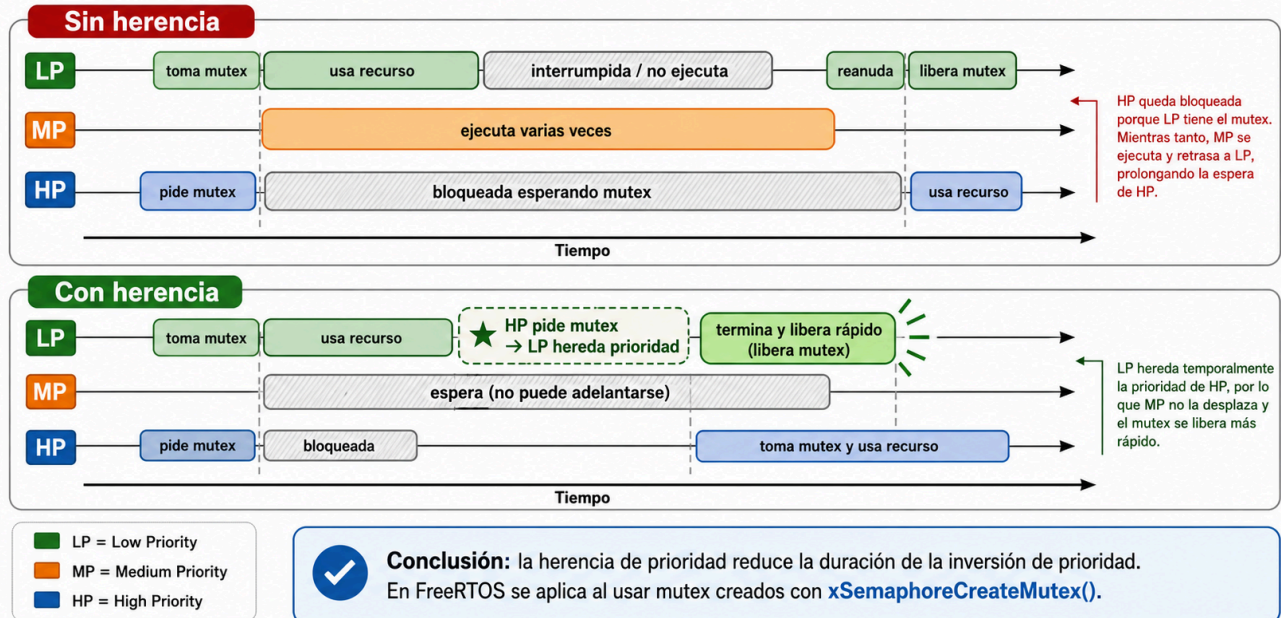
9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.

La herencia de prioridad es el mecanismo por el cual una tarea de baja prioridad que posee un mutex hereda temporalmente la prioridad de la tarea de mayor prioridad que está bloqueada esperando ese mismo mutex. Su objetivo no es eliminar conceptualmente la inversión de prioridad, sino reducir su duración.

El siguiente esquema muestra la diferencia entre no tener herencia y sí tenerla:

Herencia de prioridad

Comparación entre un sistema sin herencia y con herencia de prioridad en FreeRTOS



En FreeRTOS este mecanismo está asociado a los mutex creados con `xSemaphoreCreateMutex()`. Por eso, cuando el objetivo es proteger un recurso compartido entre tareas, conviene usar un mutex y no un semáforo binario genérico. Aun así, la mejor práctica sigue siendo diseñar secciones críticas cortas y evitar depender de la herencia como solución principal.

10. Punto muerto

10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Un punto muerto o deadlock ocurre cuando dos o más tareas quedan esperando indefinidamente recursos que están retenidos entre sí. El caso es: la Tarea A toma el Mutex X y espera el Mutex Y; la Tarea B toma el Mutex Y y espera el Mutex X. Ninguna puede continuar ni liberar lo que posee.

El siguiente ejemplo usa dos tareas, una cola de comandos y dos recursos del microcontrolador: LCD e I2C. Está escrito para mostrar el error de diseño: las tareas toman los mutex en distinto orden.

```
typedef enum { CMD_REFRESH, CMD_LOG } CmdId_t;

typedef struct
{
    CmdId_t id;
} Cmd_t;

static QueueHandle_t xCmdQueue;
static SemaphoreHandle_t xLCDMutex;
static SemaphoreHandle_t xI2CMutex;

static void vTaskA(void *pvParameters)
{
    Cmd_t cmd;
```

```

for(;;)
{
    if(xQueueReceive(xCmdQueue, &cmd, portMAX_DELAY) == pdPASS)
    {
        /* Orden A: primero LCD, después I2C. */
        xSemaphoreTake(xLCDMutex, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(10));
        xSemaphoreTake(xI2CMutex, portMAX_DELAY);

        lcd_refresh_over_i2c();

        xSemaphoreGive(xI2CMutex);
        xSemaphoreGive(xLCDMutex);
    }
}

static void vTaskB(void *pvParameters)
{
    for(;;)
    {
        /* Orden B: primero I2C, después LCD. Esto puede provocar deadlock. */
        xSemaphoreTake(xI2CMutex, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(10));
        xSemaphoreTake(xLCDMutex, portMAX_DELAY);

        lcd_write_log_over_i2c();

        xSemaphoreGive(xLCDMutex);
        xSemaphoreGive(xI2CMutex);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void app_main(void)
{
    Cmd_t start = { CMD_REFRESH };

    xCmdQueue = xQueueCreate(4, sizeof(Cmd_t));
    xLCDMutex = xSemaphoreCreateMutex();
    xI2CMutex = xSemaphoreCreateMutex();

    xTaskCreate(vTaskA, "TaskA", 2048, NULL, 2, NULL);
    xTaskCreate(vTaskB, "TaskB", 2048, NULL, 2, NULL);

    xQueueSendToBack(xCmdQueue, &start, 0);
}

```

Secuencia posible: 1) TaskA recibe un comando de la cola y toma xLCDMutex. 2) TaskB toma xI2CMutex. 3) TaskA intenta tomar xI2CMutex, pero lo tiene TaskB, entonces se bloquea. 4) TaskB intenta tomar xLCDMutex, pero lo tiene TaskA, entonces se bloquea. Ambas quedan bloqueadas para siempre.

La forma correcta de evitarlo es imponer un orden global de toma de recursos, por ejemplo siempre tomar primero xI2CMutex y después xLCDMutex en todas las tareas, o rediseñar el sistema para que un único I2C_Manager_Task sea el dueño exclusivo del bus I2C y el resto de tareas se comuniquen con él mediante colas.